

DAT42-3

Machine Learning III — Unsupervised Models

Albert School — 22 hours

Preface

In the two machine-learning courses behind you, a label told you when you were right. In DAT42-1 you built decision trees, random forests, and gradient boosting machines, and every one of them learned by comparing its prediction against a known answer — a churn flag, a house price, a fraud label. Cross-validation, feature importance, the whole apparatus of model comparison rested on one quiet luxury: a column of ground truth you could score against.

This course removes that column. In **unsupervised learning** there is no label, no answer key, no score that settles an argument. You are handed a table of customers, or documents, or sensor readings, and asked to find structure that nobody has written down — groups that hang together, directions along which the data varies, a two-dimensional picture of a fifty-dimensional cloud. The algorithms that do this are not hard to run; `scikit-learn` fits a k-means model in one line. The difficulty is everywhere else. **Which** grouping is the right one? There is no f-score to consult. **How many** groups are there? The data will not tell you; you must decide, and defend the decision. And once you have a clustering, what does it **mean** for the business — is “cluster 3” a real customer segment worth a marketing budget, or an artefact of where you happened to set a parameter?

This is why the course’s tagline insists that the burden of meaning shifts to **you**. A supervised model can be wrong in a way the data proves. An unsupervised model can only be **useful** or **useless**, and that verdict is a matter of judgement informed by metrics, never settled by them. We will treat every algorithm — k-means, hierarchical clustering, DBSCAN, PCA, t-SNE — as half technical method and half business framing. You will learn to run each one, to know exactly when it breaks, to evaluate a result you cannot score against truth, and finally to turn a cloud of points into a written report a manager can act on. That last skill, not the algorithms, is the point of the course.

Contents

Preface	2
1 The Unsupervised Setting	4
1.1 Structure without an answer key	4
1.2 Distance is the hidden assumption	4
2 K-means Clustering	5
2.1 The idea, before the algorithm	5
2.2 Choosing k : the elbow method	6
2.3 Choosing k : the silhouette score	7
3 When K-means Breaks	7
3.1 The assumptions hiding in the algorithm	7
4 Hierarchical Clustering	9
4.1 Building a tree instead of guessing k	9
5 DBSCAN	10
5.1 Clusters as regions of high density	10
6 Principal Component Analysis	11
6.1 Too many columns to see or to trust	11
6.2 How much did you keep? The explained variance ratio	12
7 t-SNE for Visualisation	12
7.1 A map built to preserve neighbourhoods, not distances	12
8 Evaluating a Clustering Without Truth	13
8.1 Scoring the unscoreable	13

9 From Algorithm to Business Decision	14
9.1 The framing problem comes first	14
9.2 One last fork: visualisation versus downstream modelling	16

1 The Unsupervised Setting

1.1 Structure without an answer key

Start with a concrete table. A retailer hands you one row per customer with two columns: **annual spend** and **number of visits per year**. Nobody has labelled these customers; there is no “segment” column. Yet when you plot the points, your eye immediately does something the supervised courses never asked of it — it sees **groups**. A knot of frequent low-spenders here, a sparse scatter of rare big-ticket buyers there. No label told you they were different; the geometry did.

Unsupervised learning is the formalisation of that act of seeing.

Definition 1 (Supervised vs unsupervised learning) In **supervised** learning each record carries a **target** label y , and the model learns a mapping from features x to y that can be scored against held-out truth. In **unsupervised** learning there is **no** target: the data is a set of feature vectors $x_1, \dots, x_n$ alone, and the goal is to discover structure within them — **clusters** of similar records, or a lower-dimensional space that preserves their important variation — with no ground truth to score against.

The missing label changes everything downstream. There is no train/test split that proves you right, because there is nothing to predict. Two analysts can produce two different clusterings of the same customers and **both** be defensible. The question is never “is this correct?” but “is this **useful**, and can I justify it?”

Worked example — The same data, two honest answers. Give the retailer’s two-column table to two analysts. One finds **three** clusters — budget, mainstream, premium — and the marketing team builds three campaigns around them. The other finds **five**, splitting premium into “loyal premium” and “occasional splurge”. Neither is wrong. The three-cluster view is simpler to act on; the five-cluster view captures a distinction that might matter for retention. Which to ship is a **business** decision about what the company can actually do differently for each group — not a number the algorithm computes.

1.2 Distance is the hidden assumption

Every clustering algorithm in this course rests on a notion of how **far apart** two records are. Change the distance and you change the clusters. The default — and the one k-means hard-codes — is ordinary Euclidean distance.

Definition 2 (Euclidean distance) For two records $x = (x_1, \dots, x_p)$ and $z = (z_1, \dots, z_p)$ with p numeric features, the **Euclidean distance** is

$$d(x, z) = \sqrt{\sum_{j=1}^p (x_j - z_j)^2}.$$

It is the straight-line distance in p -dimensional space, and it treats every feature as contributing on the same footing.

That last phrase hides a trap that will wreck a clustering before any algorithm runs. “On the same footing” means a difference of 1 counts equally in every feature — but features rarely share units.

Common pitfall **Unscaled features let one column dominate the distance.** Suppose annual spend ranges over 0–10000 (euros) and visits over 0–50. In the Euclidean formula a €1000 spend gap contributes 1000^2 while a 50-visit gap contributes only 50^2 — spend outweighs visits by a factor of four hundred. The clustering will group purely by spend and ignore visits entirely, not because visits do not matter but because of their **units**. Almost always, **standardise** every feature (subtract the mean, divide by the standard deviation — the `StandardScaler` of DAT42-1) before clustering, so each contributes comparably. Forgetting to scale is the single most common unsupervised-learning mistake.

Remark Scaling was optional for the tree ensembles of DAT42-1: a decision tree splits one feature at a time and does not care about units. Distance-based methods are the opposite — scaling is not a tweak, it is part of **defining the problem**. The same data, scaled differently, is a different question.

2 K-means Clustering

2.1 The idea, before the algorithm

Return to the retailer’s customers, now standardised. We **believe** there are k groups; suppose $k = 3$. Each group can be summarised by a single representative point — its **centroid**, the average customer of that group. If we knew the three centroids, assigning each customer would be trivial: send them to the nearest centroid. And if we knew the assignment, computing the centroids would be trivial: average each group. The catch is that we know **neither** — so k-means guesses one, deduces the other, and repeats until the two stop disagreeing.

Definition 3 (K-means objective) Given n points and a chosen number of clusters k , k-means seeks k centroids $\mu_1, \dots, \mu_k$ and an assignment of each point to a cluster that minimises the **within-cluster sum of squares** (also called **inertia**)

$$\text{WCSS} = \sum_{i=1}^k \sum_{x \in C_i} d(x, \mu_i)^2,$$

the total squared distance from every point to its own cluster’s centroid. Small inertia means tight, compact clusters.

Definition 4 (Lloyd's algorithm (the k-means iteration))

1. **Initialise** k centroids (e.g. at k random data points).
2. **Assign** each point to its nearest centroid.
3. **Update** each centroid to the mean of the points now assigned to it.
4. **Repeat** steps 2–3 until assignments stop changing.

Each round can only lower inertia or leave it unchanged, so the algorithm always converges — though not necessarily to the best possible clustering.

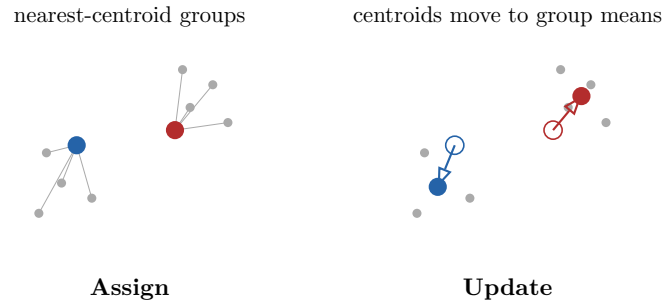


Figure 1: One iteration of Lloyd’s algorithm. **Assign** (left): each point joins its nearest centroid. **Update** (right): each centroid (filled) moves to the mean of its points, leaving its old position (hollow). Repeating these two steps drives the centroids to the centres of the groups.

Worked example — Reading a converged clustering as customer segments. Run k-means with $k = 3$ on the standardised retailer data and it returns three centroids. Translating them back to original units (un-standardising) gives the **cluster profile** — the average customer of each group:

Cluster	Avg spend (€)	Avg visits/yr	Size
0 — Budget regulars	320	28	1 940
1 — Mainstream	1 450	12	3 110
2 — Premium rare	5 600	3	410

Only now does the analysis begin. Cluster 0 visits often but spends little — candidates for an upsell. Cluster 2 spends heavily but rarely appears — worth a concierge touch to raise frequency. The **centroids are numbers; the segments are a story you tell about them**, and that story is the deliverable.

2.2 Choosing k : the elbow method

K-means cannot choose k for you — you must hand it the number of clusters. Pick too few and you merge distinct groups; too many and you shatter one group into meaningless slivers. The **elbow method** offers a first, rough guide.

Definition 5 (Elbow method) Fit k-means for a range of k (say 1 to 10) and plot the resulting inertia (WCSS) against k . Inertia always falls as k rises — more centroids fit the data more tightly — but the **rate** of improvement usually drops sharply once k passes the natural number of groups. The “elbow”, where the curve bends from steep to shallow, suggests a sensible k .

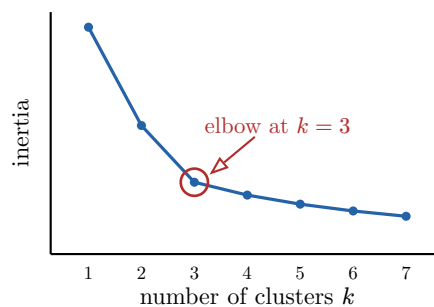


Figure 2: An elbow plot. Inertia drops steeply up to $k = 3$, then flattens: beyond three clusters each extra centroid buys little. The bend suggests $k = 3$.

Common pitfall The elbow is often **not** a clean corner. Real data frequently produces a smoothly bending curve with no obvious kink, and three analysts will point to three different k values. Treat the elbow as a **suggestion**, never a verdict, and always corroborate it with the silhouette score below and — decisively — with whether the resulting clusters make business sense.

2.3 Choosing k : the silhouette score

The elbow looks only at compactness. The **silhouette** looks at something better: whether each point sits more comfortably in its own cluster than in the next-best one.

Definition 6 (Silhouette coefficient) For a point i , let $a(i)$ be its mean distance to the other points in its **own** cluster, and $b(i)$ its mean distance to the points of the **nearest other** cluster. The silhouette coefficient is

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}, \quad s(i) \in [-1, 1].$$

A value near +1 means the point is far from neighbouring clusters (well placed); near 0 means it sits on a boundary; **negative** means it is closer to another cluster than its own (probably misassigned). The **silhouette score** of a clustering is the average of $s(i)$ over all points; the k giving the highest average is a strong candidate.

Worked example — When silhouette overrules the elbow. On a dataset the elbow plot leaves ambiguous between $k = 3$ and $k = 4$, you compute the average silhouette: 0.61 at $k = 3$, 0.48 at $k = 4$. The drop signals that the fourth cluster was carved out of points that sit on a boundary — splitting a coherent group rather than revealing a new one. You choose $k = 3$. Two imperfect signals, read together, gave a decision neither gave alone.

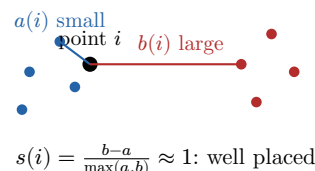


Figure 3: The silhouette compares $a(i)$, the average distance to a point's own cluster, with $b(i)$, the average distance to the nearest other cluster. When $b(i) \gg a(i)$ the point is comfortably placed and $s(i)$ approaches 1.

Choosing k . Run the elbow method for a quick range-finding, confirm with the silhouette score, and let **business interpretability** break ties — the best k is one whose clusters you can name and act on. No single number decides it.

3 When K-means Breaks

3.1 The assumptions hiding in the algorithm

K-means is fast and intuitive, which makes it easy to forget how much it assumes. Every assumption is a failure case waiting for the wrong dataset.

Definition 7 (What k-means quietly assumes)

1. **You know k** in advance.
2. Clusters are **roughly spherical** (because “nearest centroid” with Euclidean distance carves space into straight-edged cells).
3. Clusters are of **similar size and spread** (one shared notion of “near” applies everywhere).
4. Clusters are **convex and linearly separable** — no rings, no crescents.
5. Every point **belongs to some cluster**: k-means has no concept of an outlier and will force noise into the nearest group.

Worked example — Failure 1 — two crescents (non-convex shapes). Take the classic “two moons”: two interleaving crescent shapes. A human sees two clusters instantly. K-means, forced to draw a straight boundary between two centroids, slices both crescents in half and returns two clusters that each contain part of **both** moons — a confident, fast, and completely wrong answer. The shape is non-convex, and k-means cannot represent it.

Worked example — Failure 2 — unequal spread and density. Now two clusters: a tight, dense ball of 500 points and, beside it, a broad diffuse cloud of 500 points. Because k-means applies one uniform distance everywhere, its boundary sits too close to the diffuse cloud, stealing the cloud’s outer points into the dense ball. Differing **spread** defeats it. The same happens with very **unequal sizes**: a cluster of 50 points next to one of 5000 is often swallowed whole.

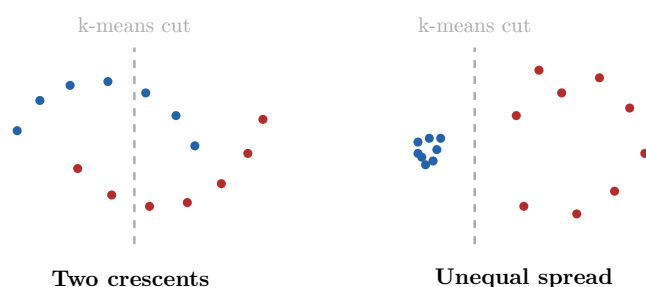


Figure 4: Two datasets where k-means fails. Left: interleaving crescents — the straight centroid boundary splits each crescent across both clusters. Right: a dense ball beside a diffuse cloud — the uniform boundary cuts into the cloud and misassigns its near edge. Colour shows the **true** groups, not k-means’ output.

Common pitfall K-means **always returns an answer**, and the answer always looks plausible — k tidy groups, each with a centroid. Plausibility is not correctness. Before trusting a k-means result, ask whether your clusters could be non-spherical, unequal in density, or noisy; if so, reach for the methods in the next two chapters. The danger of k-means is precisely that it never complains.

4 Hierarchical Clustering

4.1 Building a tree instead of guessing k

K-means demands k up front. **Agglomerative hierarchical clustering** refuses to: it starts with every point as its own cluster and repeatedly merges the two closest clusters, recording the whole history as a tree. You choose k **after** seeing that tree, not before.

Definition 8 (Agglomerative hierarchical clustering)

1. Begin with each of the n points as a singleton cluster.
2. Find the two **closest** clusters and merge them.
3. Repeat until a single cluster remains.

The sequence of merges, drawn as a tree, is a **dendrogram**. Closeness between two clusters is set by a **linkage** rule — e.g. **single** (distance between nearest members), **complete** (farthest members), or **average** (mean over all pairs).

Definition 9 (Dendrogram) A **dendrogram** is a tree whose leaves are the individual points. Each horizontal bar joins two clusters, drawn at a **height** equal to the distance at which they merged. Low joins unite very similar points; high joins unite already-large, dissimilar groups. **Cutting** the dendrogram with a horizontal line at some height yields a clustering: each branch crossing the line becomes one cluster.

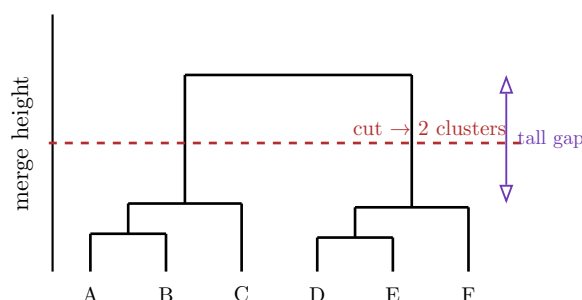


Figure 5: A dendrogram of six points. Cutting where the vertical gap between successive merges is **largest** (red line) yields two clusters, $\{A,B,C\}$ and $\{D,E,F\}$. The tall gap means the two groups are far more distant from each other than their members are from one another.

Worked example — Reading a dendrogram to pick the number of clusters. You are not told how many segments exist. You build a dendrogram and look for the **tallest vertical gap** — the largest jump in merge height. In the figure, the merges within each trio happen at heights below 1, but uniting the two trios requires height 2.6: a long unbroken stretch with no merge. Cutting anywhere in that gap gives two clusters and says, quantitatively, “these two groups are far more unlike each other than anything inside them.” The number of clusters fell out of the data’s structure rather than being assumed.

Common pitfall The **linkage rule changes the tree**. Single linkage merges on the nearest pair and tends to produce long straggly “chains”; complete linkage merges on the farthest pair and favours compact balls; average linkage sits between. The same data can yield different dendrograms — and different cut points — under different linkages. State which linkage you used; it is part of the result, not a hidden default.

Remark Hierarchical clustering computes distances between every pair of points, so its cost grows roughly with n^2 (or worse). It is illuminating on thousands of points and a dendrogram you can actually read, but impractical on millions — a case where you fall back to k-means or to DBSCAN.

5 DBSCAN

5.1 Clusters as regions of high density

Both methods so far are blind to two things real data is full of: clusters of **arbitrary shape**, and **noise**. DBSCAN — Density-Based Spatial Clustering of Applications with Noise — addresses both by redefining what a cluster **is**. Not “a ball around a centroid” but “a connected region where points are packed densely”, however that region is shaped, with sparse points left over as noise.

Definition 10 (DBSCAN) DBSCAN takes two parameters: a radius ε and a count `minPts`. A point is a **core point** if at least `minPts` points lie within distance ε of it. A cluster grows by chaining together core points that are within ε of each other, sweeping in the non-core points (**border points**) on their fringe. Any point reachable from no core point is labelled **noise** and belongs to no cluster.

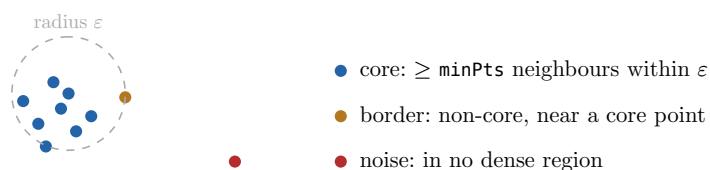


Figure 6: DBSCAN’s three roles. A **core** point (blue) has at least `minPts` neighbours within ε ; cores within ε of each other link into one cluster. A **border** point (amber) is non-core but lies inside a core’s neighbourhood. A **noise** point (red) sits in no dense region and joins no cluster.

Worked example — Where DBSCAN wins and k-means lost. Hand DBSCAN the two crescents that defeated k-means. Each crescent is a connected ridge of dense points, so DBSCAN chains along it and recovers **both** moons whole, curve and all — it never needed them to be round. Sprinkle in twenty scattered outliers and DBSCAN does what k-means cannot: it labels them **noise** and leaves them out, instead of jamming each into the nearest group and distorting that group’s profile.

Worked example — When DBSCAN itself struggles. DBSCAN is not universal. Give it clusters of genuinely **different densities** — one tight, one loose — and a single $(\varepsilon, \text{minPts})$ cannot suit both: a radius large enough to connect the loose cluster will merge the tight one with its neighbours, while a radius tuned to the tight cluster shatters the loose one into noise. Density-varying data is DBSCAN’s blind spot, just as shape was k-means’.

Common pitfall DBSCAN’s results swing sharply with ε . Too large and separate clusters fuse into one blob; too small and almost everything is labelled noise. There is no k to set, but ε must still be chosen with care — a common heuristic is to plot each point’s distance

to its k -th nearest neighbour, sort, and pick ε at the “knee” of that curve. Do not accept the first ε you try.

Three clustering tools, three temperaments. **K-means** — fast, needs k , assumes round equal clusters, no noise concept. **Hierarchical** — no k up front, yields an interpretable dendrogram, costly on big data. **DBSCAN** — finds arbitrary shapes and isolates noise, needs no k , but struggles when densities vary. Match the tool to the shape of your data, not your habit.

6 Principal Component Analysis

6.1 Too many columns to see or to trust

Clustering met its limits in two dimensions; real data has hundreds. A customer table might carry 200 behavioural features, a gene-expression table 20000. You cannot plot 200 dimensions, and worse, distance-based methods **degrade** in high dimensions as points drift toward equidistance. **Dimensionality reduction** finds a handful of new directions that retain most of the data’s variation, so you can see it, cluster it, or model it in a far smaller space.

PCA is the foundational method. Its idea is geometric and worth meeting before any algebra.

Worked example — Finding the direction that matters. Plot height against weight for a population: the cloud is a tilted ellipse, stretched along a diagonal. Neither height nor weight alone captures it — but a single new axis running **along** the ellipse’s long diagonal (call it “overall size”) captures almost all the spread, while the short perpendicular axis (“build”, thin vs stocky at a given size) captures the little that remains. PCA finds exactly these axes automatically: it rotates to the directions along which the data varies **most**.

Definition 11 (Principal components) The **principal components** of a dataset are an ordered set of orthogonal directions: the **first** is the direction of **maximum variance** in the data; the **second** is the direction of greatest remaining variance perpendicular to the first; and so on. They are the eigenvectors of the data’s covariance matrix, and each one’s eigenvalue is the variance the data has along it. Projecting points onto the first few components reduces dimension while preserving as much variance as possible.

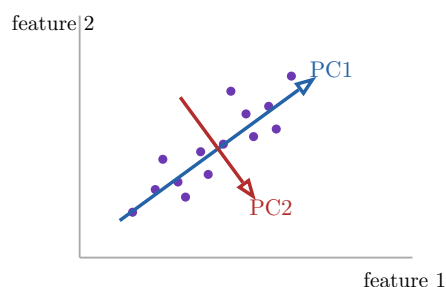


Figure 7: PCA on a tilted cloud. **PC1** (blue) points along the direction of greatest variance; **PC2** (red), perpendicular to it, captures the smaller remaining spread. Describing each point by its position along PC1 alone keeps most of the information in one dimension instead of two.

6.2 How much did you keep? The explained variance ratio

Reducing dimensions throws information away. PCA lets you measure exactly how much you kept, so the trade-off is explicit rather than blind.

Definition 12 (Explained variance ratio) Each principal component accounts for a fraction of the data's total variance: its eigenvalue divided by the sum of all eigenvalues. This is its **explained variance ratio**. The ratios are decreasing (PC1 explains the most) and sum to 1 across all components. The **cumulative** ratio over the first m components tells you what fraction of the original variation survives a reduction to m dimensions.

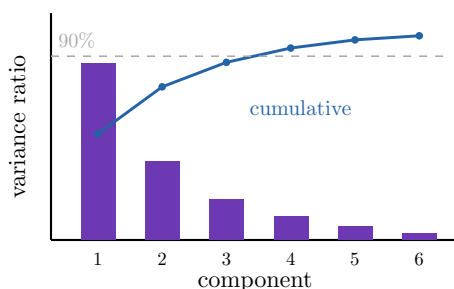


Figure 8: A scree plot. Bars give each component's explained variance ratio (decreasing); the blue line is the cumulative total. Here the first four components already exceed 90% of the variance, so reducing from six dimensions to four costs under 10% of the information.

Worked example — Compress 200 features to 10. A 200-feature customer table is too wide to cluster reliably. You run PCA, plot the cumulative explained variance, and find the first 10 components retain 85% of the variance. You cluster in that 10-dimensional space instead: distances behave better, k-means runs faster, and you have lost only 15% of the variation — a trade you can state precisely and defend.

Common pitfall PCA is not feature selection. It does not pick 10 of your 200 columns; it builds 10 brand-new axes, each a weighted blend of **all** originals. A principal component has no native name — “PC1” might be $0.3 \times \text{spend} + 0.4 \times \text{visits} - 0.2 \times \text{returns} + \dots$. You can sometimes interpret it by inspecting which original features load most heavily on it, but you can no longer say “we used the spend column”; you used a combination. And because each component mixes every feature, PCA destroys interpretability that a tree's feature importances gave you for free.

Common pitfall PCA maximises **variance**, and variance lives in a feature's units — so an unscaled feature with a large numeric range will dominate the first component for no reason but its scale. **Standardise before PCA**, exactly as before clustering, or PC1 will simply track whatever column happens to be measured in the largest numbers.

7 t-SNE for Visualisation

7.1 A map built to preserve neighbourhoods, not distances

PCA reduces dimension by preserving **global** variance — it is a rigid rotation. But to **see** a high-dimensional dataset as a 2-D picture, what you usually want is for **neighbours to stay**

neighbours: points close in 200-D should land close on the page, even if the global geometry must be bent to achieve it. That is what t-SNE (t-distributed Stochastic Neighbour Embedding) is built for, and it produces the strikingly clustered scatter plots you have seen of image or text datasets.

Definition 13 (t-SNE) t-SNE is a **non-linear** dimensionality-reduction method for **visualisation**. It converts pairwise similarities in the high-dimensional space into probabilities (nearby points have a high chance of being “neighbours”), then arranges points in 2-D so that the low-dimensional neighbour-probabilities match the high-dimensional ones as closely as possible. The result excels at revealing **local cluster structure**: groups that are genuinely separate tend to appear as visually distinct islands.

Worked example — Seeing clusters PCA blurred. A 64-dimensional dataset of handwritten-digit images, projected to 2-D by PCA, looks like one overlapping smear — its clusters lie along curved manifolds that a linear projection flattens together. The **same** data through t-SNE separates into ten clean islands, one per digit. For the narrow job of **seeing** that the groups exist, t-SNE succeeds where PCA’s global, linear view cannot.

Common pitfall t-SNE distances are not interpretable — only neighbourhoods are. The method is engineered to preserve who-is-near-whom, and it deliberately **sacrifices** global geometry to do so. Three traps follow, and every one is a real mistake made on real t-SNE plots:

- **Cluster sizes mean nothing.** t-SNE expands dense clusters and contracts sparse ones; a big blob is not a bigger or more important group.
- **Distances between clusters mean nothing.** Two islands sitting close together are not more similar than two far apart — the gaps are not to scale.
- **Apparent clusters can be artefacts of the perplexity parameter.** Different perplexities yield different pictures; clusters that appear at one setting can dissolve at another.

Read a t-SNE plot as “these points form a local group”, never as “this group is this big” or “these two groups are this far apart”.

Remark These cautions are **why** t-SNE is for the eye, not the pipeline. Because its output coordinates carry no faithful distance, you must not feed them into k-means or use them as features for a downstream model — a point the final chapter makes general. t-SNE answers “do separable groups appear to exist?” and nothing more.

8 Evaluating a Clustering Without Truth

8.1 Scoring the unscorable

In DAT42-1 evaluation was a solved problem: hold out labelled data, compute accuracy or RMSE. Here there are no labels, so the very meaning of “good” must be manufactured. **Internal** metrics try to do so using only the geometry of the clusters — how **compact** each is and how **separated** they are from one another.

Definition 14 (Silhouette score (as an evaluation metric)) The average silhouette coefficient (Chapter 2) over all points, in $[-1, 1]$. Higher is better: it rewards clusterings that are simultaneously **tight** (small $a(i)$) and **well separated** (large $b(i)$). It needs no labels and can compare different k or different algorithms on the same data.

Definition 15 (Davies–Bouldin index) For every pair of clusters, form the ratio of their combined internal scatter to the distance between their centroids; for each cluster take its worst (largest) such ratio with any other; average those over all clusters. This is the **Davies–Bouldin index**, in $[0, \infty)$. **Lower is better** — small values mean clusters are compact and far apart. Note the direction is opposite to the silhouette’s: high silhouette and **low** Davies–Bouldin both signal a good clustering.

Worked example — Two metrics agreeing on the same number of clusters. Clustering the retailer data at $k = 2, 3, 4, 5$, you tabulate both metrics:

k	2	3	4	5
silhouette ($\uparrow$ better)	0.53	0.61	0.49	0.42
Davies–Bouldin ($\downarrow$ better)	0.78	0.62	0.81	0.95

Silhouette peaks and Davies–Bouldin bottoms out at the **same** $k = 3$. When two internal metrics, built on different principles, agree, you can choose $k = 3$ with more confidence than either gave alone.

Common pitfall Internal metrics measure geometry, not meaning — and they are biased. Both silhouette and Davies–Bouldin reward **compact, well-separated, roughly spherical** clusters, which is precisely the shape k-means produces. So they systematically **flatter k-means** and **penalise** the elongated or density-based clusters that DBSCAN correctly finds — a perfect crescent clustering can score **worse** than a wrong spherical one. A high silhouette does not mean the clustering is **true** or **useful**; it means it is round. Never let an internal metric be the final word.

Remark When you **do** have a few labelled examples — a sample hand-tagged by an expert — **external** metrics (such as the adjusted Rand index) compare your clusters against those labels and are far more trustworthy than internal ones. They are a luxury unsupervised learning rarely affords, but worth seeking: even a small labelled sample turns an unanswerable question into a partly answerable one. The deepest evaluation, though, remains whether the clusters **do something** for the business.

9 From Algorithm to Business Decision

9.1 The framing problem comes first

Every chapter so far solved a technical task. The hardest step bookends them and uses no algorithm at all: deciding **whether and how** a business question is an unsupervised one, and

turning the output back into something a non-technical manager can act on. This is where most real projects succeed or fail.

Definition 16 (Framing a problem as unsupervised) A question is well posed for unsupervised learning when (1) there is **no label** to predict — you are looking for structure, not forecasting a known target — and (2) the structure you find would **change a decision**. “Which customers are about to churn?” is supervised (a churn label exists). “Are there natural customer segments we should serve differently?” is unsupervised — **if** the company can in fact treat segments differently. If it cannot, the cleanest clustering in the world is useless.

Worked example — A complete unsupervised project, end to end. Question. Marketing asks: “Should we run one campaign or several?”

1. **Frame.** No label exists; segments would change the campaign plan; marketing can run up to four distinct campaigns. A clustering with $k \leq 4$ is actionable. The framing already bounds the technical work.
2. **Prepare.** Assemble per-customer features (spend, frequency, recency, categories). **Standardise** them.
3. **Reduce (optional).** 80 features $\rightarrow$ PCA to 12 components retaining 88% variance, so distances behave.
4. **Cluster.** k-means across $k = 2..6$; elbow suggests 3–4, silhouette peaks at 3, Davies–Bouldin agrees. Choose $k = 3$.
5. **Profile and name.** Un-standardise the centroids: “Budget regulars”, “Mainstream families”, “Premium occasional”.
6. **Report.** Hand marketing not a scatter plot but three named segments, each with its size, its defining behaviour, and **one recommended action**.

The algorithm occupied one step. Framing, preparation, choosing k , and translation occupied the other five — and carried the value.

Definition 17 (Actionable cluster description) A cluster description is **actionable** when it states, in business language and without reference to the algorithm: the segment’s **size** (how many customers, what share of revenue), its **defining characteristics** (what makes it distinct, in real units), a **memorable name**, and a **recommended action** the business can actually take. “Cluster 2, centroid (1.8, −0.4, ...)” is not a description; “Premium occasional — 6% of customers but 22% of revenue; spend heavily, visit rarely; action: a concierge programme to lift visit frequency” is.

Common pitfall The commonest failure of an unsupervised project is **delivering clusters instead of decisions** — handing over a coloured scatter plot and cluster IDs and calling it done. A manager cannot act on “cluster 3”. Always close the loop: name each segment, describe it in the business’s own terms, and attach a recommended action. An un-interpreted clustering is the unsupervised equivalent of a number with no answer — technically present, practically worthless.

9.2 One last fork: visualisation versus downstream modelling

Dimensionality reduction served two very different masters in this course, and conflating them is a subtle, costly error.

Definition 18 (Two uses of dimensionality reduction) **For visualisation:** reduce to 2–3 dimensions purely to **look** at the data and judge by eye whether structure exists. Faithful distances are not required, only a readable picture — this is t-SNE’s domain (and PCA’s, for a quick linear view).

For downstream modelling: reduce dimension to feed the result into another algorithm — clustering, or a supervised model — so that the new coordinates must **preserve real geometry** (distances, variance). This is PCA’s domain; its components are stable, distance-respecting features.

Common pitfall **Never feed t-SNE coordinates into a downstream model.** Because t-SNE distorts distances and cluster sizes by design (Chapter 7), its 2-D output is a **picture**, not a **feature space**. Clustering on t-SNE coordinates, or using them as model inputs, builds on distances the method itself warns are meaningless — and the artefacts of **perplexity** leak straight into your model. Use **PCA** (or another distance-preserving method) when the reduced data must be **modelled**; reserve **t-SNE** for when it will only be **looked at**. Choosing the wrong tool here quietly corrupts everything built on top of it.

Course takeaways. Unsupervised learning trades the safety of a label for the burden of judgement. **Scale first** — distance-based methods live or die by it. **Cluster** with the method that fits your data’s shape: k-means for round, equal groups (choose k by elbow + silhouette + business sense); hierarchical for an interpretable dendrogram and no preset k ; DBSCAN for arbitrary shapes and noise. Know each one’s failure cases, because none of them ever complains. **Reduce dimension** with PCA when geometry must survive (and you can read the explained variance you keep), with t-SNE only to **see** structure — never to model it, and never reading its distances or sizes literally. **Evaluate** with internal metrics (silhouette $\uparrow$, Davies–Bouldin $\downarrow$) while remembering they merely reward roundness, not truth or usefulness. And **finish the job**: turn clusters into named, sized, actionable segments a manager can act on. Without a label to be right against, being **useful** — and being able to justify it — is the whole of the work.

Exercises

1. Take any two-feature dataset on very different scales (e.g. income in euros and age in years). Cluster it with k-means **before** and **after** standardising, and show how the unscaled clustering groups almost entirely by the larger-ranged feature. Explain the mechanism in terms of the Euclidean distance formula.
2. Implement k-means on a dataset with three visible groups. Plot the elbow curve over $k = 1..8$ and the average silhouette over $k = 2..8$, choose a k from the two together, and translate the chosen centroids into a profile table with one business sentence per cluster.
3. Construct or load two datasets where k-means fails — one with non-convex shapes (e.g. two moons), one with strongly unequal cluster spread or size. Show k-means’ wrong output on each and explain which specific assumption it violates.

4. Run agglomerative hierarchical clustering on a small dataset and plot the dendrogram. Identify the tallest vertical gap, cut there, and state how many clusters result and why that gap justifies the choice. Repeat with a different linkage rule and describe how the tree changes.
5. Apply DBSCAN to the two-moons dataset with added scattered outliers. Show that it recovers both crescents and labels the outliers as noise. Then vary ε to demonstrate both failure modes — everything fused into one cluster, and almost everything labelled noise.
6. On a high-dimensional dataset (≥ 20 features), run PCA, plot the explained variance ratio per component and its cumulative curve, and report how many components retain 90% of the variance. Inspect the loadings of PC1 and attempt a business interpretation of what it represents.
7. Project the same high-dimensional dataset to 2-D with both PCA and t-SNE and compare the plots. Then re-run t-SNE at two different perplexity values and show how the apparent clusters change — using this to argue why t-SNE distances and cluster sizes must not be read literally.
8. Evaluate a clustering with both the silhouette score and the Davies–Bouldin index across a range of k . Where they agree, justify the chosen k ; then find or construct a case (e.g. DBSCAN on crescents) where a **correct** non-spherical clustering scores **worse** on these internal metrics than a wrong spherical one, and explain the bias.
9. Take a genuine business question, argue whether it is properly an unsupervised task (no label, and the structure would change a decision the business can act on), carry it through preparation, optional PCA, clustering, and choice of k , and deliver a one-page written report of named, sized, actionable cluster descriptions — no algorithm jargon, one recommended action per segment.
10. For one reduced dataset, state explicitly whether your reduction is for visualisation or for downstream modelling, justify your choice of t-SNE versus PCA accordingly, and explain what would go wrong if you swapped them.